

GrAND: Supplementary Material

A Algorithms for FreshDiskANN-GPU

FreshDiskANN-GPU maintains three types of indices: a Long-Term Index (LTI) containing the consolidated base graph, M read-only temporary indices holding up to T recently inserted points each, and one active read-write index with capacity T for new insertions. Since only the read-write index is modified, locks are required only for that region.

Insert: The insert procedure is detailed in Algorithm 1. Insertions follow the standard FreshDiskANN procedure. We first reserve slots for all points in the batch and initialize their graph entries. Each point independently performs *BANGSearch* from the read-write index medoid to obtain a candidate neighbor set, after which *GrANDRobustPrune* selects at most $\mathcal{R}$ outgoing edges. Once outgoing neighbors are fixed, reverse edges are added to the neighboring vertices. Since only vertices in the read-write index can be modified, reverse-edge updates are protected using fine-grained per-vertex locks, while the rest of the graph remains lock-free. Existing reverse edges are left unchanged. If a neighbor has already reached its degree limit, *GrANDRobustPrune* is invoked again to determine its updated adjacency list.

Algorithm 1: FreshDiskANN-GPU InsertBatch

Input: Batch $\mathcal{B} = \{(id_i, x_i)_{i=1}^B\}$; beam width ℓ ; pruning parameter α , degree bound $\mathcal{R}$

```

1 foreach  $(id_i, x_i) \in \mathcal{B}$  in parallel do
2   Assign slot  $p_i \leftarrow rwBase + rwCount + i$ 
3   Write  $x_i$  into entry  $p_i$ 
4    $deg(p_i) \leftarrow 0$ 
5   Map  $id_i$  to  $p_i$ 
6 foreach  $(p_i, x_i) \in \mathcal{B}$  in parallel do
7    $V_i \leftarrow \text{BANGSearch}(G, x_i, medoid_{RW}, \ell_i)$ 
8 foreach  $p_i$  in parallel do
9    $N_{out}(p_i) \leftarrow \text{GrANDRobustPrune}(p_i, V_i \setminus \{p_i\}, \alpha, \mathcal{R})$ 
10  $E \leftarrow \{(u, p_i) \mid u \in N_{out}(p_i)\}$ 
11 foreach  $(u, p) \in E$  where  $p \notin N_{out}(u)$  in parallel do
12   Lock( $u$ )
13   if  $|N_{out}(u)| < \mathcal{R}$  then
14     Append  $p$  to  $N_{out}(u)$ 
15   else
16      $N_{out}(u) \leftarrow \text{GrANDRobustPrune}(u, N_{out}(u) \cup \{p\}, \alpha, \mathcal{R})$ 
17   Unlock( $u$ )

```

Search: The search procedure is described in Algorithm 2. The LTI index, each temporary index, and the read-write index function as independent subgraphs with their own medoids. Each index is searched independently from its own medoid using the *BANGSearch* procedure. Since edges never cross index boundaries, each search explores only a single subgraph. The top- k candidates returned from every index are then merged, and the globally closest k non-deleted points are returned as the final result.

Delete: Deletions use a delete bitset of size D . Deleted points are filtered during search, and when the bitset reaches capacity, the a merge operation is triggered to remove tombstoned points and repair neighborhoods via exact consolidation.

Merge: Once the read-write index reaches its capacity, it is sealed and becomes a read-only temporary index. When the maximum number of temporary indices is reached, a background **merge** consolidates them into the LTI index while removing deleted points. The merge procedure is described in Algorithm 3.

The procedure first drains pending updates and allocates a new graph. In the first phase, active vertices from the existing LTI index are copied into the new graph, and their neighborhoods are repaired by removing deleted neighbors and reapplying *RobustPrune*. In the second phase, all active vertices from the temporary indices are inserted into the new LTI

Algorithm 2: FreshDiskANN-GPU SearchBatch

Input: Query set Q ; result size k ; beam width ℓ
Output: Set of nearest neighbors K

```

1  $I \leftarrow \{(LTI, medoid_{LTI})\}$  // List of indices and medoids
2 for each temporary index  $T$  do
3    $I \leftarrow I \cup \{(T, medoid_T)\}$ 
4 if read-write index is not empty then
5    $I \leftarrow I \cup \{(RW, medoid_{RW})\}$ 
6 foreach  $(i, m) \in I$  do
7   foreach  $q \in Q$  in parallel do
8      $W_q \leftarrow \text{BANGSearch}(G, x_q, m, \ell)$ 
9      $topk_{i,q} \leftarrow$  first  $k$  non-deleted points in  $W_q$ 
10 foreach  $q \in Q$  in parallel do
11    $result_q \leftarrow k$  closest elements of  $\bigcup_i topk_{i,q}$ 
12 return  $\{result_q\}_{q \in Q}$ 

```

index using the standard insertion procedure. Finally, the active read-write index is copied unchanged, and metadata is updated to reflect the enlarged LTI and empty temporary-index set.

Algorithm 3: FreshDiskANN-GPU StreamingMerge

Input: Graph G ; pruning parameter α ; degree bound $\mathcal{R}$; beam width ℓ
Output: Graph G'

```

1 Drain operation stream
2 Allocate new graph  $G'$  of the same capacity
// Phase A: Removal of deleted points with neighborhood repair
3 foreach  $v \in LTIIndex \cup TempIndices$  in parallel do
4   if  $deleted(v)$  then
5     Skip this point
6   Copy vector( $v$ ) to  $G'$ 
7    $C \leftarrow \{n \in N_{out}(v) : n \text{ is not deleted}\}$ 
8   Sort  $C$  by distance from  $v$ 
9    $N_{out}(v) \leftarrow \text{GrANDRobustPrune}(v, C, \alpha, \mathcal{R})$ 
// Phase B: Insertion of new points
10 foreach  $p \in TempIndices$  in parallel do
11   if  $deleted(p)$  then
12     Skip this point
13    $N_{out}(p) \leftarrow \text{InsertBatch}(G', x_p, m_{LTI}, \ell)$ 
14 Copy read-write index from  $G$  to  $G'$ 
15 Let  $t$  be the number of active temporary indices
16  $|LTIIndex| \leftarrow |LTIIndex| + t \cdot T$ 
17 Set number of temporary indices to 0
18 Atomically swap graph pointers
19 Clear DeleteList
20 Free the old graph

```
